

CONSTRUIRE DANS LES RÈGLES DE L'ART

Application de suivi d'entraînement — retour d'expérience, architecture cible, modèle de données, neuf phases de construction et décisions à trancher.

V1 → V2
DOCUMENT
DE CADRAGE

SOMMAIRE

Comment utiliser ce document	0
Retour d'expérience de la v1	1
Objectifs et périmètre	2
Architecture cible	3
Modèle de données	4
Sécurité et conformité	5
Phases de construction	6
Décisions à trancher	7
Qualité et méthode	8
Reprise des données existantes	9
Message d'amorçage	10

0 — COMMENT UTILISER CE DOCUMENT

Il sert de point de départ à une conversation dédiée, et de référence pendant toute la construction.

Ce cahier ne décrit pas une application à écrire d'un bloc. Il décrit une **méthode** et un **ordre**. Chaque phase produit un livrable vérifiable avant d'ouvrir la suivante — c'est ce qui distingue une application construite d'un prototype qui s'étend jusqu'à devenir ingérable.

La dernière partie contient un message à copier tel quel pour ouvrir la conversation de développement. Le reste sert de référence commune : quand une question surgit en cours de route, elle a normalement sa réponse ici, ou elle figure dans la liste des décisions à trancher.

Ce document assume un choix structurant : la v2 abandonne le chiffrement de bout en bout côté client au profit d'une base relationnelle avec contrôle d'accès côté serveur. La partie 5 explique pourquoi, et ce que cela coûte. C'est la première décision à valider ou à rejeter — tout le reste en découle.

1 — RETOUR D'EXPÉRIENCE DE LA V1

Ce qui a été construit, ce qui a fonctionné, et les six murs rencontrés.

La v1 est une application web mono-fichier, hébergée sur GitHub Pages, utilisée sur iPhone. Elle contient un programme d'entraînement de 23 machines, un carnet de séances, une analyse par séance, un bilan global, une gestion de photos, un capteur cardiaque Bluetooth, un chiffrement local et une synchronisation vers un dépôt GitHub privé.

Ce qui a fonctionné : la vitesse de construction, le fait que tout tienne dans un fichier autonome, le rendu identique sur tous les navigateurs WebKit, et la qualité du carnet lui-même — saisie, chronomètre, rappel de la performance précédente, analyse graphique.

Les six murs, tous instructifs :

MUR	ENSEIGNEMENT
Safari n'ouvre pas un fichier HTML local (iOS 18) : Coup d'œil affiche la page mais n'exécute pas le JavaScript.	Sur iOS, une application web doit être hébergée. Il n'y a pas de mode « fichier local ».
WebKit n'implémente pas le Web Bluetooth. Chrome et Firefox sur iOS sont des Safari déguisés.	Tout accès matériel sur iPhone passe par une app native ou un navigateur tiers spécialisé.
Bluefy ne sait pas écrire de fichiers : les exports n'atterrissent nulle part.	Ne jamais faire reposer la sauvegarde sur un téléchargement navigateur.
Safari efface le stockage local après 7 jours sans ouverture (ITP).	Le stockage navigateur est un cache, jamais une source de vérité.
Le chiffrement client interdit toute requête : impossible de filtrer, trier ou agréger côté serveur.	Chiffrement de bout en bout et exploitation des données sont largement exclusifs.
Le programme est codé en dur dans le fichier.	Ce qui varie doit être une donnée, jamais du code.

Ces six points ne sont pas des échecs : ce sont les contraintes réelles de la plateforme, découvertes par la pratique. Ils justifient à eux seuls l'architecture de la v2.

2 — OBJECTIFS ET PÉRIMÈTRE

Ce que la v2 doit faire, et ce qu'elle ne fera pas.

OBJECTIFS

- **Fiabilité des données** — aucune perte possible, sauvegarde et restauration gérées par le service.
- **Exploitabilité** — pouvoir interroger l'historique librement, aujourd'hui et dans dix ans.
- **Multi-utilisateur** — comptes séparés, isolation stricte, inscription autonome.
- **Programmes comme données** — créer, modifier, versionner, dupliquer, importer, partager.
- **Qualité professionnelle** — migrations versionnées, tests, environnements séparés, conformité.

PÉRIMÈTRE FONCTIONNEL

Repris de la v1 : catalogue d'exercices avec schémas, fiches techniques et fiches produit ; carnet complet (séries, charge, reps, RIR, horaires, cardio, abdos, poids du corps) ; capteur cardiaque avec courbe échantillonnée ; analyse de séance ; bilan global ; photos de machines.

Nouveau : comptes utilisateurs ; éditeur de programmes (séances, jours de repos, ordre des exercices, séries et repos par exercice) ; versionnement d'un programme sans casser l'historique ; bibliothèque d'exercices extensible par l'utilisateur ; fiches machines rattachées à une salle, avec QR code ; import et export dans un format ouvert et documenté.

HORS PÉRIMÈTRE EXPLICITE

- **Écriture dans l'app Santé d'Apple** — impossible depuis le web, quelle que soit l'architecture. Passerelle par export uniquement.
- **Application native iOS** — à réévaluer plus tard si le capteur devient central pour des clients.
- **Réseau social, coaching, paiement** — hors sujet à ce stade.

3 — ARCHITECTURE CIBLE

Un client web, une base managée, aucun serveur applicatif à écrire.

COUCHE	CHOIX PROPOSÉ	RAISON
Base de données	PostgreSQL managé (Supabase), région européenne	Relationnel, requêtable, sauvegardes gérées, contrôle d'accès dans la base
Authentification	Fournie par le service (email + mot de passe, magic link)	Ne jamais écrire soi-même une authentification
Autorisation	Row Level Security, politiques SQL par table	Appliquée par la base, donc incontournable depuis le client
Front	Application web, à trancher : sans framework ou avec	Voir partie 7
Hébergement du front	Hébergeur statique avec HTTPS et déploiement continu	GitHub Pages convient encore, alternatives possibles
Capteur cardiaque	Web Bluetooth, donc Bluefy sur iOS	Contrainte plateforme, inchangée
Hors ligne	Cache local + file d'attente, synchronisation à la reconnexion	La salle peut être sans réseau ; le local redevient un cache

Le point d'architecture le plus important : le client ne doit jamais être la garantie de sécurité. Toute règle qui compte — un utilisateur ne voit que ses données, on ne modifie pas une séance d'un autre — s'écrit dans la base, pas dans le JavaScript.

4 — MODÈLE DE DONNÉES

Première proposition, à challenger en début de projet.

```
profiles      id · user_id · pseudo · unites · cree_le
exercices     id · proprietaire · nom · tag · zone · muscles ·
              technique · erreurs · alternatives · schema_svg · public
programs      id · proprietaire · nom · description · version ·
              parent_id · statut · public
program_days  id · program_id · ordre · type (seance | repos) ·
              code (J1...) · titre · cible · echauffement · cardio
program_items id · program_day_id · ordre · exercise_id ·
              series · reps · repos_s · consigne · unilateral
sessions      id · proprietaire · program_id · program_day_id ·
              date · heure_debut · heure_fin · poids_corps ·
              fc_moy · fc_max · fc_min · note
session_sets  id · session_id · exercise_id · ordre_exercice ·
              numero_serie · charge · reps · rir · debut_le · fin_le
session_cardio id · session_id · moment (debut | fin) · machine ·
              programme · duree_min · niveau · distance_km ·
              calories · fc_moy · note
session_abs   id · session_id · exercice · note · secondes · reps · charge
hr_samples    id · session_id · t_offset_s · bpm
clubs         id · nom · adresse
machines      id · club_id · exercise_id · marque · modele ·
              qr_contenu · emplacement · photo_url
```

Trois principes portés par ce schéma. Le programme est séparé de son exécution : modifier un programme ne réécrit pas l'historique, puisqu'une séance référence la version du jour où elle a été faite. Une série est une ligne, pas un objet enfoui dans un JSON — c'est ce qui rend l'analyse possible en SQL. Et les exercices comme les machines sont des données partageables entre utilisateurs, avec un indicateur de visibilité.

La courbe cardiaque en lignes individuelles représente environ 360 lignes par séance, soit 90 000 lignes par an et par utilisateur — négligeable pour PostgreSQL, et cela permet des requêtes par plage de temps. Une alternative en tableau compressé est possible si le volume devenait un sujet.

5 — SÉCURITÉ ET CONFORMITÉ

Le sujet où « les règles de l'art » ont le plus de contenu concret.

L'ARBITRAGE SUR LE CHIFFREMENT

La v1 chiffre côté client : l'hébergeur ne peut techniquement rien lire. C'est la protection la plus forte, et elle interdit toute requête. La v2 propose le modèle standard des applications professionnelles : **TLS en transit, chiffrement au repos** assuré par l'hébergeur, et **contrôle d'accès strict**. L'hébergeur devient un tiers de confiance, encadré contractuellement.

Une voie mixte existe et mérite d'être étudiée : chiffrer côté client uniquement les champs libres — notes personnelles, photos — et laisser en clair les données numériques nécessaires aux calculs. Complexité supplémentaire notable pour un gain réel, à arbitrer.

CE QUI DOIT ÊTRE EN PLACE DÈS LA PREMIÈRE PHASE

- **RLS activée sur toutes les tables**, sans exception, avec politiques explicites pour lecture, écriture, modification, suppression.
- **Aucun secret dans le front** : seule la clé publique du service y figure, la sécurité reposant sur les politiques.
- **Environnements séparés** : développement, préproduction, production. Ne jamais développer sur les données réelles.
- **Sauvegardes vérifiées** : une restauration testée vaut mille sauvegardes supposées.
- **Journalisation** des accès et des erreurs.

RGPD — LE SUJET À NE PAS REPOUSSER

Des données d'entraînement, de poids corporel et de fréquence cardiaque constituent des **données de santé** au sens du règlement. Cela impose : consentement explicite et documenté, minimisation, information claire sur les finalités, droit d'accès, de rectification, d'effacement et de portabilité, durée de conservation définie, et registre des traitements.

Conséquences concrètes sur le produit : hébergement en Union européenne, un export complet accessible à l'utilisateur, une suppression de compte qui efface réellement, une politique de confidentialité rédigée, et un accord de sous-traitance avec l'hébergeur. Ces éléments sont des fonctionnalités à développer, pas des formalités à ajouter à la fin.

6 — PHASES DE CONSTRUCTION

Chaque phase se termine par un livrable vérifiable. On n'ouvre pas la suivante avant.

00 Cadrage et modèle de données

1 à 2 séances

Valider ou rejeter l'arbitrage sur le chiffrage. Affiner le modèle de données, nommer les champs, définir les types et les contraintes. Écrire les migrations SQL initiales, versionnées dans le dépôt. Choisir le fournisseur et créer les environnements.

Critère de sortie : un schéma SQL appliqué en développement, relu, avec ses contraintes d'intégrité et ses index.

01 Authentification et isolation

1 séance

Inscription, connexion, réinitialisation de mot de passe, déconnexion. Politiques RLS sur une table témoin.

Test décisif : créer deux comptes et vérifier depuis le client que l'un ne peut lire ni écrire les données de l'autre, y compris en forgeant des requêtes à la main.

Critère de sortie : deux comptes hermétiques, prouvé par un test écrit et rejouable.

02 Catalogue d'exercices

1 séance

Migrer les 23 exercices existants — schémas SVG, technique, erreurs, alternatives, fiches produit — vers la base. Consultation, recherche, filtrage par zone. Création d'un exercice personnel par l'utilisateur.

Critère de sortie : le catalogue v1 entièrement en base, affiché depuis la base et non depuis le code.

03 Éditeur de programmes

2 séances

Créer un programme, y ajouter des séances et des jours de repos, ordonner les exercices, définir séries, répétitions et repos. Dupliquer un programme, le faire évoluer en créant une nouvelle version sans altérer l'historique. Importer et exporter au format documenté.

Critère de sortie : le programme actuel recréé intégralement par l'interface, sans écrire une ligne de code.

04 Carnet et saisie

2 séances

Reprendre l'ergonomie de la v1, qui a fait ses preuves : saisie série par série, ajout et retrait de séries, chronomètre de repos, rappel de la performance précédente, blocs cardio et abdos, horaires automatiques. Écriture directe en base, avec cache local et file d'attente pour le mode hors réseau.

Critère de sortie : une séance complète saisie en salle sans réseau, retrouvée intacte après reconnexion.

05 Capteur cardiaque

1 séance

Connexion Bluetooth, courbe échantillonnée, horodatage automatique des exercices pour la corrélation, stockage des points en base.

Critère de sortie : une séance réelle avec courbe complète et fréquence moyenne par exercice.

06 Analyse et bilan

2 séances

Analyse de séance et bilan global, cette fois calculés par des requêtes SQL et non en JavaScript. Progression par exercice, détection des stagnations, volume par période, corrélations avec le poids et la fréquence cardiaque.

Critère de sortie : des requêtes documentées, réutilisables, dont les résultats sont vérifiés à la main sur un échantillon.

07 Machines, salles et QR

1 séance

Fiches machines rattachées à une salle, rattachées à un exercice. Saisie du contenu d'un QR code, emplacement, marque et modèle. Photos associées.

Critère de sortie : les machines de Taverny renseignées et affichées dans les fiches d'exercice.

08 Conformité et mise en production

1 à 2 séances

Politique de confidentialité, consentement, export complet, suppression de compte effective, durée de conservation. Environnement de production, sauvegardes vérifiées par une restauration réelle, surveillance des erreurs.

Critère de sortie : un compte créé, utilisé, exporté puis supprimé de bout en bout, avec preuve d'effacement.

Sur l'estimation. Une « séance » désigne une session de travail substantielle, pas une heure. L'ordre importe davantage que la durée : chaque phase repose sur la précédente, et sauter le cadrage se paie toujours au triple en phase 4.

7 — DÉCISIONS À TRANCHER

À arbitrer en début de projet. Aucune n'a de réponse évidente.

Chiffrement de bout en bout ou base exploitable ?

Conserver le chiffrement client interdit les requêtes, le multi-utilisateur véritable et la récupération de mot de passe. L'abandonner confie la lisibilité à l'hébergeur.

Recommandation : abandonner le chiffrement intégral, envisager le chiffrement des seuls champs libres. C'est la pratique professionnelle courante, et elle seule permet les objectifs annoncés.

Avec ou sans framework front ?

Sans framework : peu de dépendances, contrôle total, mais tout est à écrire. Avec un framework moderne : composants, routage et état gérés, au prix d'une chaîne de compilation et d'une dépendance durable.

Recommandation : adopter un framework dès lors que l'application devient multi-écrans et multi-utilisateur. La v1 a montré les limites du tout-manuel dès que le fichier dépasse quelques centaines de kilo-octets.

Application web ou application native à terme ?

Le web reste bridé sur iOS : pas de Bluetooth hors Bluefy, pas d'accès à Santé, stockage limité. Le natif lève ces limites mais impose un autre métier et le circuit App Store.

Recommandation : rester web pour la v2. Réévaluer si des clients exigent capteurs ou intégration Santé — dans ce cas, une application native s'imposera.

Quel format d'échange pour les programmes ?

Un format propriétaire est plus rapide à écrire ; un format documenté et versionné permet l'import, l'export, le partage et la reprise ultérieure.

Recommandation : définir dès la phase 3 un schéma JSON documenté et versionné. C'est ce qui rendra les programmes partageables entre utilisateurs, et exploitables plus tard.

Utilisateur unique dès le départ, ou multi d'emblée ?

Ajouter le multi-utilisateur après coup impose de reprendre chaque table et chaque requête.

Recommandation : multi-utilisateur dès la phase 1, même avec un seul compte réel. Le coût initial est faible, le coût rétroactif est majeur.

8 — QUALITÉ ET MÉTHODE

Ce qui distingue une application professionnelle d'un prototype étendu.

- **Migrations versionnées** — chaque évolution du schéma est un fichier SQL numéroté, dans le dépôt, rejouable sur un environnement vierge. Jamais de modification manuelle en production.
- **Tests** — au minimum sur les politiques d'accès et les calculs d'analyse. Un test qui échoue vaut mieux qu'un bug découvert en salle.
- **Environnements séparés** — développement, préproduction, production, avec des données distinctes.
- **Déploiement continu** — pousser sur la branche principale déclenche le déploiement, après passage des tests.
- **Journal des décisions** — un fichier qui consigne chaque choix d'architecture et sa raison. Inestimable six mois plus tard.
- **Documentation vivante** — modèle de données, requêtes d'analyse, procédure de restauration.
- **Surveillance** — remontée des erreurs client et serveur, sinon les bugs ne se découvrent qu'en salle.

Sur les tests, un conseil issu de la v1 : plusieurs bugs sérieux — perte de la séance en cours, écran de verrouillage invisible — sont passés parce que les tests vérifiaient la logique interne et non ce que l'utilisateur voit réellement. Tester l'état d'une variable ne suffit pas ; il faut tester l'écran.

9 — REPRISE DES DONNÉES EXISTANTES

La v1 contient un historique réel à ne pas perdre.

Les données de la v1 sont accessibles par l'export JSON de l'onglet Diagnostic, ou par les fichiers du dépôt de synchronisation. Structure d'une séance :

```
{ id, date, day, t1, t2, bw, note,
  c1: { mach, prog, min, niv, km, kcal, fc, note },
  c2: { ... },
  entries: [ { k: "chest-press", sets: [ { w, r, rir } ] } ],
  abs:      [ { ex, note, sets: [ { sec, r, w } ] } ],
  hr: { avg, max, min, src }, hrs: [ [t_s, bpm] ], hr0, st: { ... } }
```

La migration consiste à éclater cette structure vers le modèle relationnel : une ligne par série, une ligne par point cardiaque, une ligne par bloc cardio. À écrire comme un script de migration rejouable, pas comme une opération manuelle.

10 — MESSAGE D'AMORÇAGE

À copier tel quel pour ouvrir la conversation de développement.

Je veux construire la version 2 d'une application de suivi d'entraînement, dans les règles de l'art. J'ai un cahier de reprise que je te transmets.

Contexte : j'ai développé une v1 fonctionnelle mais bricolée – application web mono-fichier, données chiffrées en local, synchronisation vers un dépôt GitHub. Elle marche, mais elle bute sur quatre besoins : fiabilité des données, exploitation de l'historique, multi-utilisateur, et programmes d'entraînement modifiables au lieu d'être codés en dur.

Ma démarche est aussi un apprentissage : je veux comprendre et appliquer les pratiques professionnelles, pour pouvoir ensuite proposer des applications à des clients sur d'autres sujets.

Commençons par la phase 00 : valider l'arbitrage sur le chiffrement, puis construire le modèle de données et les migrations SQL initiales.

Avant de coder quoi que ce soit, challenge mon modèle de données et mes choix d'architecture. Dis-moi ce qui te paraît fragile.

Joins ce document à la conversation, ainsi qu'un export JSON de tes données actuelles si tu veux traiter la migration tôt. Et garde ce réflexe qui a bien servi sur la v1 : demander systématiquement ce qui pourrait mal tourner avant de valider une étape.

Ce document est un point de départ, pas un contrat. Le modèle de données et les phases doivent être challengés en début de projet — c'est précisément l'objet de la phase 00.

Rédigé à partir de l'expérience réelle de la v1 : ses réussites, ses six murs, et ses bugs instructifs.